Mid-way presentation

Hackathon Management Platform

Nicolas Wagener · Fredrik Spratling · Israel Micheal
Enriko Kroj · Daniel Ivanov

Project scope

01

Publish

Organizers publish challenges and deadlines

02

Form teams

Students form teams

03

Register

Teams register for a challenge

04

Submit

Teams submit before the deadline

05

Evaluate

Judges score; organizers publish results

One platform supports the complete hackathon lifecycle.

Stakeholders & needs

Stakeholder	Category	RE role	Needs
Participant	Primary	Provider / validator	Find team, see instructions, submit projects.
Organiser	Primary	Provider / validator / decision maker	Configure challenges, rules and deadlines; manage registrations; publish results.
Judge	Primary	Provider / validator	See and score submissions.
University	Secondary	Decision maker	Host the event.
IT support / Admin	Hidden	Maintainer / operator	Manage accounts and backups.

User stories

Participant

“Create a team or join one through an invite code, so we don’t coordinate over email.”

Judge

“Declare a conflict of interest, so I’m not asked to grade a team I know.”

Organizer

“Set and, if needed, extend deadlines, so I can react to problems during the event.”

Admin

“Automated backups of submissions, so no team loses work.”

Functional requirements

FR1	Users register and log in securely.	All roles
FR2	Role-based access: participant, organizer, judge, admin.	All roles
FR3	Create, edit and search challenges, including rules and team size.	Organizers
FR4	Set and extend registration and submission deadlines.	Organizers
FR5	Create teams, invite members with code/link, leave a team.	Participants
FR6	Register a team for a challenge.	Participants
FR7	Upload/update until deadline; then lock the submission.	Participants
FR8	Assign judges; judges can declare a conflict of interest.	Organizers / judges
FR9	Score with configurable rubric and written feedback.	Judges
FR10	Aggregate scores into a ranking; organizer publishes it.	Organizers / participants

Non-functional requirements

Security

Passwords stored securely; judges see only assigned submissions; uploads scanned.

Performance

Responsive under the expected peak submission workload.

Reliability

Time-stamped receipt for successful submissions; daily backups; logging.

Usability

New participant joins a team and registers in under 5 minutes; mobile-friendly.

Maintainability

Modular codebase, technical documentation and automated testing.

Conflicts

Participants vs fairness	Teams want last-second edits and extensions when uploads fail; other teams may see this as unfair. **OPTION · Short grace period for technical failures confirmed by logs.**
Transparency vs judge privacy	Participants want to know who scored them and why; judges may prefer anonymity. **OPTION · Show feedback without judge names.**
Organizer flexibility vs rules	Organizers may want exceptions (e.g., a six-person team) despite team-size limits. **OPTION · Allow an organizer override with a logged reason.**

Open questions & assumptions

Open questions

- Can external people join (other universities, industry judges)?
- How is the final score computed: judge average, rubric weights, normalization?
- Are late submissions allowed? Is there a grace period?
- Are results public or only visible to participants?

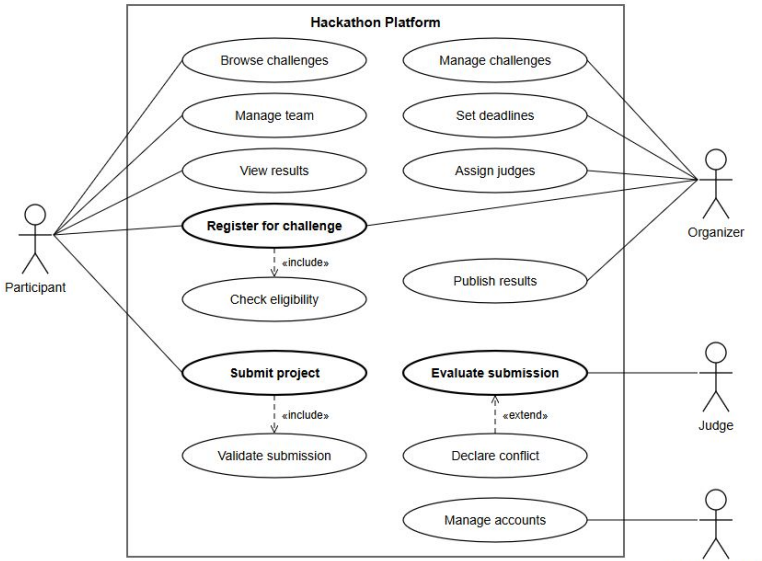
Assumptions

- One challenge per team.
- One team per participant.

User story map

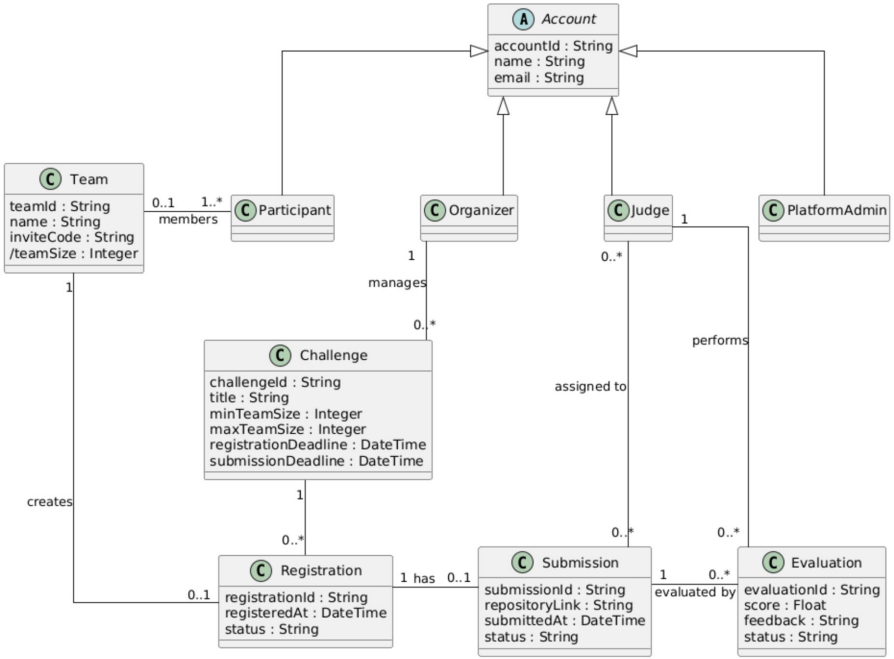
	Join event	Form a team	Register for a challenge	Submit a project	Judge & publish
TASKS	Log in; browse challenges	Create team; invite members	Pick challenge; check eligibility	Upload files; confirm receipt	Score; rank; publish
MVP	Email/password login; searchable challenge list	Create team; join via invite code	Registration button; team-size check	Upload link/files; deadline lock; receipt	Rubric scoring; automatic ranking; publish
RELEASE 1	—	Team-finder board	Waitlist when full	Edit until deadline; version history; check repo	Conflict declaration; written feedback

Use case diagram



Main use cases in bold: UC1 Submit project, UC2 Register for challenge, UC3 Evaluate submission

UML class diagram



Use case 1 — Submit project

MAIN FLOW

1. Open the team challenge page and select Submit.
2. Enter repository link, upload files and description.
3. System validates deadline, file type/size and required fields.
4. System stores submission with a timestamp.

PRECONDITIONS

Authenticated; team registered; deadline not passed.

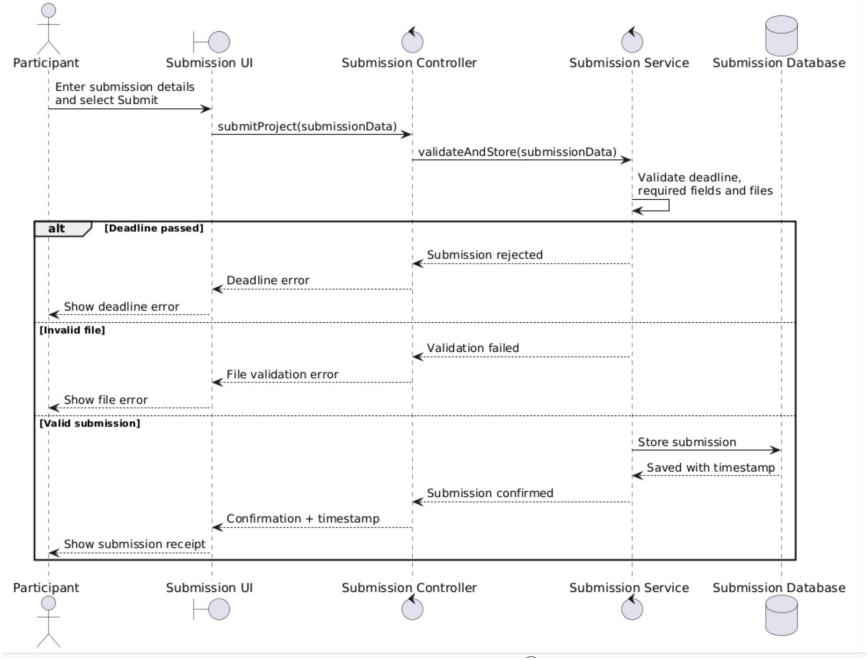
EXTENSIONS

Deadline passed → reject and show deadline.
Invalid file → show error, retain other data.
Existing submission → new active version; keep history.

POSTCONDITIONS

One active submission per team, or a clear failure reason.

UML sequence diagram: UC1



Use case 2 — Register team

MAIN FLOW

1. Browse challenges and select one.
2. Click Register team.
3. System checks team size, existing registration and capacity.
4. System confirms registration.
5. Team appears on the organizer’s list.

PRECONDITIONS

Authenticated participant; team exists; registration is open.

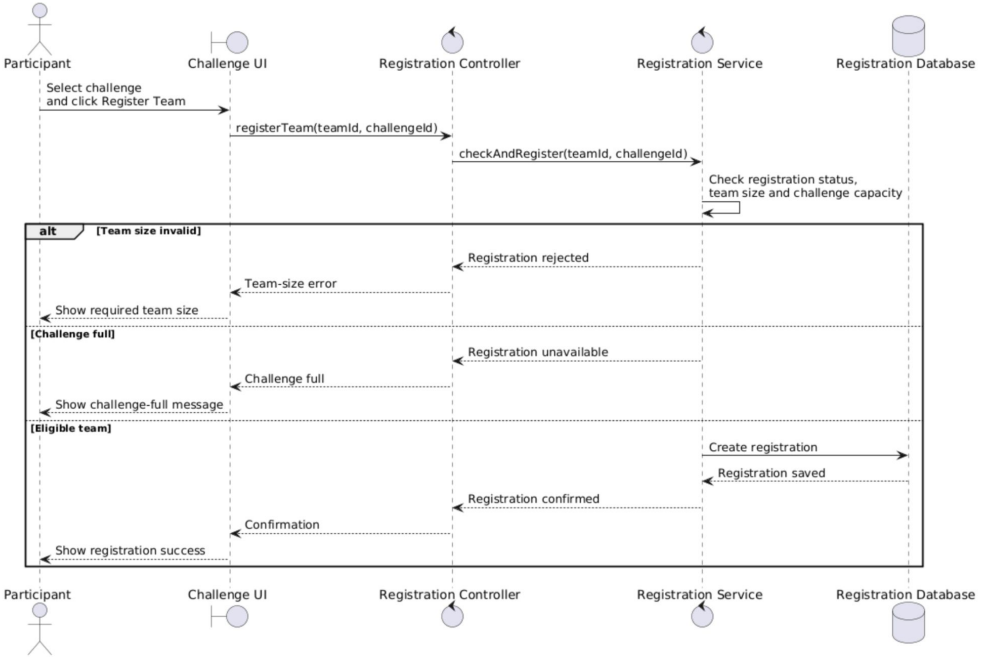
EXTENSIONS

Invalid team size → refuse, show required size.
Challenge full → offer waitlist (Release 1).

POSTCONDITIONS

Registered, waitlisted, or told why registration failed.

UML sequence diagram: UC2



Use case 3 — Evaluate submission

MAIN FLOW

1. Judge opens assigned submissions.
2. Selects submission; views files and repository link.
3. System displays evaluation rubric.
4. Judge adds score and written feedback.
5. System validates and stores evaluation.

PRECONDITIONS

Judge authenticated and assigned; submission deadline passed.

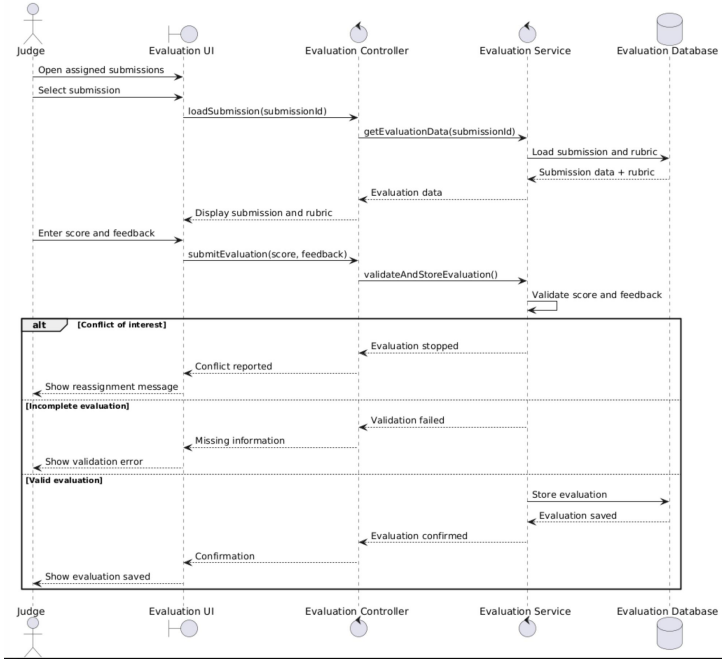
EXTENSIONS

Conflict of interest → remove assignment and notify organizer.
Incomplete evaluation → show missing score/feedback; do not save.

POSTCONDITIONS

Evaluation stored, or submission flagged for reassignment.

UML sequence diagram: UC3



Thank you.

Questions?

Hackathon Management Platform · Mid-way presentation